



Introdução ao Computador PROFCOMP

- Aula 10 –
Gerenciamento de
Processos
- Docente: Ana
Carolina Gondim
Inocêncio

Como o sistema Operacional decide quem usa a CPU



Aula 11 – Agenda

- Explicar o que é um SO
- Identificar funções
- Relacionar SO e hardware



ROTEIRO DA AULA

Agenda

01 Recordando

02 Problema Inicial

03 O que é um Processo?

04 Estados e Ciclo de Vida

05 PCB — Bloco de Controle

06 Escalonamento de Processos

07 Algoritmos de Escalonamento

08 Troca de Contexto e Threads

09 Docência

10 Atividade em Duplas



01 / 09

Recordando



ANTES DE COMEÇAR

Lembrando a Aula Anterior

- Na aula passada estudamos sobre o Sistema Operacional e sobre sua função principal de gerência, que muitas vezes passa despercebida pelo usuário.



Pergunta que fica no ar:

Como o SO decide O QUE executar, QUANDO e POR QUANTO TEMPO?



02 / 09

Problema Inicial



PROBLEMATIZAÇÃO INICIAL

Relembre o uso do seu computador

Enquanto você lê este slide, por exemplo, o seu computador provavelmente já está fazendo tudo isso ao mesmo tempo:



Editor de texto ou navegador
aberto



Música ou vídeo tocando em
segundo plano



Aplicativos de mensagem
sincronizando



Antivírus e serviços do
sistema rodando

***Pergunta: quantos núcleos de processamento (cores)
você acha que tem o seu computador?***



PROBLEMATIZAÇÃO INICIAL

Poucos Núcleos, Muitos Processos



4

núcleos é o número típico de um notebook comum

- Isso significa, no máximo, 4 instruções de processos diferentes por vez.
- Mas o Gerenciador de Tarefas mostra de 60 a 200 processos rodando ao mesmo tempo!



**Como poucos núcleos executam tantos processos
“ao mesmo tempo”?**

*É essa pergunta que o
Gerenciamento de Processos
responde.*



PROBLEMATIZAÇÃO INICIAL

A Ilusão do Paralelismo

- O processador não faz tudo de verdade ao mesmo tempo.
- Ele alterna rapidamente entre os processos, em fatias de milissegundos.
- Essa troca é tão rápida que cria a ILUSÃO de simultaneidade.

Núcleo único da CPU — linha do tempo:



“Multitarefa não é mágica — é troca rápida.”

PROBLEMATIZAÇÃO INICIAL

Então, quem decide tudo isso?



Qual processo executa agora?



Por quanto tempo ele executa?



O que acontece quando o tempo termina?

*Essa é a função do Sistema Operacional: o **Gerenciamento de Processos**.*



03 / 09

Aprofundando o conceito de Processo?



CONCEITO FUNDAMENTAL

Definição de Processo

- Processo é um programa em execução.
- Não é só o código guardado no disco — é o código + os dados + o estado atual de execução.



DEFINIÇÃO

Processo é a unidade básica de trabalho que o Sistema Operacional gerencia.



COMPARAÇÃO ESSENCIAL

Recordando Processo vs. Programa



PROGRAMA

- Passivo — é um arquivo armazenado no disco.
- Código estático (ex.: chrome.exe).
- Pode existir sem nunca ser executado.



PROCESSO

- Ativo — é a execução daquele programa.
- Tem memória, variáveis e estado atual.
- Posso abrir o mesmo programa duas vezes → dois processos diferentes.

Analogia: o programa é a receita de bolo; o processo é o ato de cozinhar.



ANATOMIA DE UM PROCESSO

O que Compõe um Processo



Espaço de Endereçamento

Memória reservada para código e dados do processo.



Contador de Programa

Indica qual instrução será executada a seguir.



Registradores da CPU

Pequenas memórias internas usadas durante a execução.



Pilha de Execução (Stack) Guarda chamadas de função e variáveis locais.



Recursos Alocados

Arquivos abertos, conexões de rede, dispositivos.



04 / 09

PCB — Bloco de Controle de Processo



PCB — PROCESS CONTROL BLOCK

O que é o PCB?

- PCB = Process Control Block (Bloco de Controle de Processo).
- É a “ficha de identidade” de cada processo dentro do Sistema Operacional.
- Guarda toda informação necessária para retomar a execução depois de uma pausa.

PROCESSO #2417
editor.exe

Estado: Pronto

PC: 0x4F2A

Prioridade: 3

Memória: 128 MB

Arquivos abertos: 2



PCB — PROCESS CONTROL BLOCK

O que Contém o PCB

PID

Identificador único do processo

Contador de Programa (PC)

Próxima instrução a executar

Escalonamento

Prioridade e informações da fila

Estado

Novo, Pronto, Executando,
Bloqueado ou Terminado

Registradores

Valores salvos da CPU no momento
da pausa

Memória

Limites e ponteiros do espaço de
endereçamento



PCB — PROCESS CONTROL BLOCK

Por que o PCB é Essencial

- Quando o SO troca de um processo para outro, ele PRECISA salvar os dados do processo atual no PCB.
- Sem o PCB, ao retomar um processo pausado, todo o progresso seria perdido.



“O PCB é a memória de curto prazo de cada processo.”



05 / 09

Estados e Ciclo de Vida



CICLO DE VIDA DO PROCESSO

Os Estados de um Processo



Novo

O processo
está sendo
criado



Pronto

Espera na fila
por uma vez
na CPU



Executando

Está usando a
CPU agora



Bloqueado

Espera um
evento (I/O,
dado, etc.)



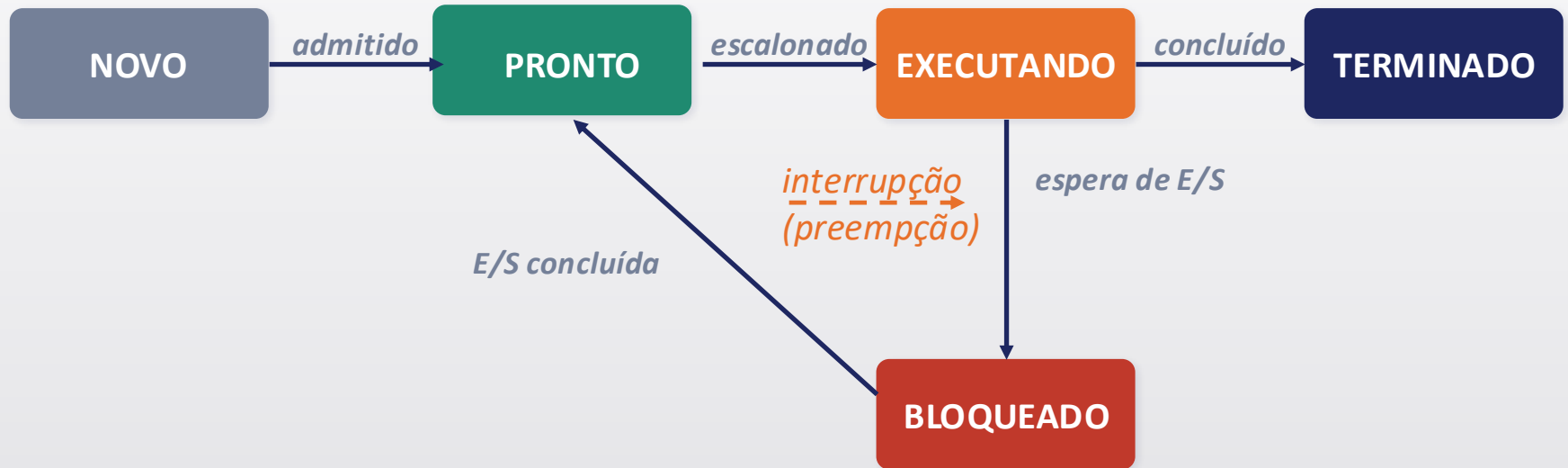
Terminado

Concluiu ou
foi finalizado



CICLO DE VIDA DO PROCESSO

Diagrama de Transição de Estados



Setas indicam as transições possíveis entre os estados de um processo.



CICLO DE VIDA DO PROCESSO

O Ciclo de Vida, Passo a Passo

1	Usuário abre o programa	<i>Processo é criado → estado NOVO</i>
2	SO aceita o processo na fila	<i>Estado passa para PRONTO</i>
3	Escalonador escolhe o processo	<i>Estado passa para EXECUTANDO</i>
4	Processo solicita leitura de arquivo	<i>Estado passa para BLOQUEADO até a E/S terminar</i>
5	E/S é concluída	<i>Estado volta para PRONTO</i>
6	Processo finaliza	<i>Estado passa para TERMINADO</i>



CICLO DE VIDA DO PROCESSO

Fila de Prontos e Fila de Bloqueados



FILA DE PRONTOS

- Reúne todos os processos que já podem executar.
- Esperam apenas a vez na CPU.
- Ordem definida pelo algoritmo de escalonamento.



FILA DE BLOQUEADOS

- Reúne processos esperando um evento externo.
- Exemplos: leitura de disco, rede, entrada do usuário.
- Só voltam à fila de prontos quando o evento ocorre.

O Sistema Operacional mantém essas filas constantemente atualizadas.



06 / 09

Escalonamento de Processos



ESCALONAMENTO DE PROCESSOS

O que é o Escalonador (Scheduler)?

- É o componente do Sistema Operacional que decide QUAL processo da fila de prontos vai usar a CPU.
- Sem ele, o sistema não saberia organizar a execução de tantos processos simultâneos.
- Ele é chamado a cada interrupção de relógio ou sempre que um processo muda de estado.



“O escalonador é o juiz que decide, a cada instante, quem joga a partida.”



ESCALONAMENTO DE PROCESSOS

Preemptivo vs. Não-preemptivo



NÃO-PREEMPTIVO

- Uma vez que o processo começa, só sai da CPU quando termina ou bloqueia.
- Mais simples de implementar.
- Pode causar longas esperas se um processo demorar muito.



PREEMPTIVO

- O SO pode interromper o processo no meio da execução para dar a vez a outro.
- Garante mais responsividade ao sistema.
- É o modelo usado pela maioria dos sistemas modernos.



ESCALONAMENTO DE PROCESSOS

CrITÉrios de Escalonamento



Utilização da CPU

Manter a CPU sempre ocupada, sem tempo ocioso.



Throughput

Quantidade de processos concluídos por unidade de tempo.



Tempo de Espera

Tempo total que o processo passa na fila de prontos.



Tempo de Turnaround

Tempo total desde a chegada até a conclusão do processo.



Tempo de Resposta

Tempo até a primeira resposta do processo ao usuário.



07 / 09

Algoritmos de Escalonamento



ALGORITMO 1 DE 3

FCFS — First Come, First Served

- O processo que chega primeiro, executa primeiro.
- Simples como uma fila de banco: ninguém “fura” a fila.
- Não-preemptivo: uma vez iniciado, o processo roda até o fim.

Exemplo — quatro processos chegam juntos (tempo 0):

Processo	Chegada	Duração	Término	Espera
P1	0	8 ms	—	—
P2	0	4 ms	—	—
P3	0	9 ms	—	—
P4	0	5 ms	—	—

Ordem de chegada: P1 → P2 → P3 → P4



ALGORITMO 1 DE 3

FCFS — Calculando o Tempo de Espera



Processo	Chegada	Duração	Término	Espera
P1	0	8	8	0
P2	0	4	12	8
P3	0	9	21	12
P4	0	5	26	21

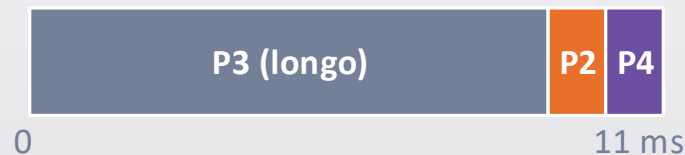
Tempo médio de espera = $(0 + 8 + 12 + 21) / 4 = 10,25$ ms



ALGORITMO 1 DE 3

O Problema do FCFS: Efeito Comboio

- Se um processo muito longo chega primeiro, todos os outros esperam muito tempo — mesmo sendo rápidos.
- Isso é chamado de efeito comboio (convoy effect): processos curtos ficam “presos” atrás de um processo longo.



Mesmo que P2 e P4 sejam rapidíssimos, eles esperam o processo longo terminar.

Solução possível: priorizar processos mais curtos → algoritmo SJF.



ALGORITMO 2 DE 3

SJF — Shortest Job First

- Escolhe sempre o processo com MENOR tempo de execução restante.
- Reduz o tempo médio de espera do sistema como um todo.
- Usando os mesmos processos, reordenamos por duração:

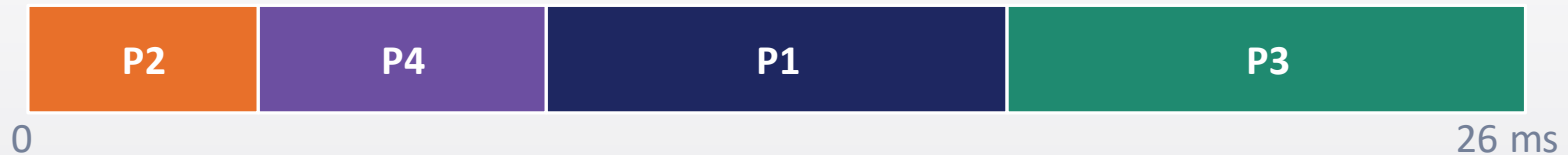
Processo	Chegada	Duração	Término	Espera
P2	0	4 ms	—	—
P4	0	5 ms	—	—
P1	0	8 ms	—	—
P3	0	9 ms	—	—

Nova ordem de execução: P2 → P4 → P1 → P3 (do menor para o maior)



ALGORITMO 2 DE 3

SJF — Calculando e Comparando



Processo	Chegada	Duração	Término	Espera
P2	0	4	4	0
P4	0	5	9	4
P1	0	8	17	9
P3	0	9	26	17

MÉDIA DE ESPERA

FCFS: 10,25 ms

SJF: 7,5 ms

SJF foi mais eficiente em média

Tempo médio de espera (SJF) = $(0 + 4 + 9 + 17) / 4 = 7,5$ ms



ALGORITMO 3 DE 3

Round Robin

- Cada processo recebe uma fatia de tempo fixa: o quantum.
- Quando o **quantum** termina, o processo volta para o FIM da fila de prontos.
- É o algoritmo preemptivo mais usado em sistemas de uso geral (desktops, celulares).



Quantum é a fatia máxima de tempo que cada processo pode usar por vez — geralmente entre 10 e 100 milissegundos.



ALGORITMO 3 DE 3

Round Robin — Simulação (quantum = 4 ms)



Processo	Chegada	Duração	Término	Espera
P1	0	8	20	12
P2	0	4	8	4
P3	0	9	26	17
P4	0	5	25	20

Tempo médio de espera = $(12 + 4 + 17 + 20) / 4 = 13,25$ ms

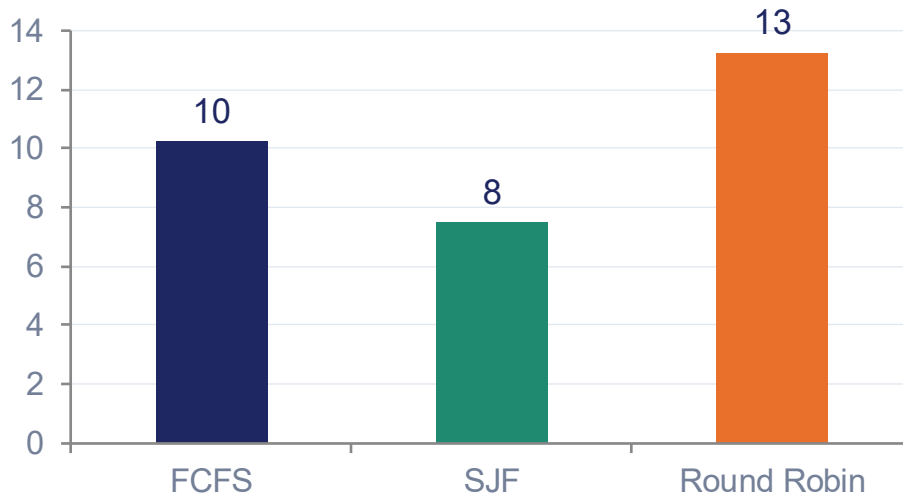
O Round Robin nem sempre tem a menor espera média — seu objetivo é a responsividade (resposta rápida a todos), não o menor tempo total.



SÍNTESE

Comparando os Três Algoritmos

Tempo médio de espera (mesmo conjunto de processos)



FCFS

Simple, mas sofre com o efeito comboio.

SJF

Menor espera média, mas exige saber a duração antecipadamente.

Round Robin

Mais justo e responsivo, mesmo sem o menor tempo de espera.



08 / 09

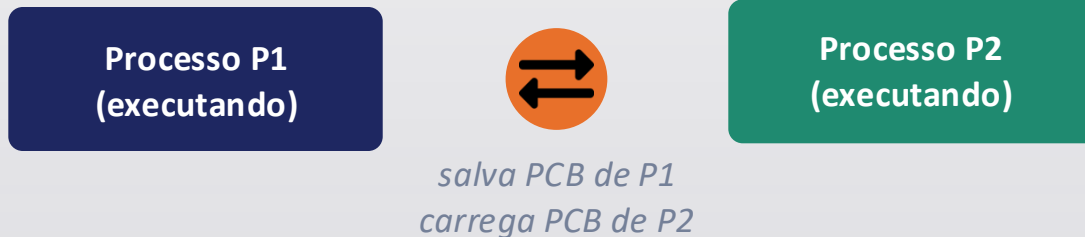
Troca de Contexto e Threads



TROCA DE CONTEXTO

O que é a Troca de Contexto?

- É o processo de salvar o estado de um processo (no PCB) e carregar o estado de outro.
- Tem um custo (overhead): durante a troca, a CPU não produz trabalho útil para nenhum processo.
- Trocar contexto demais — quantum muito pequeno — desperdiça tempo de CPU.



Quanto menor o quantum, mais trocas — e mais tempo “perdido” apenas trocando.



CONCEITO COMPLEMENTAR

Processos vs. Threads



PROCESSO

- Possui seu próprio espaço de memória, isolado dos demais.
- Mais “pesado” para criar e trocar de contexto.
- Falha em um processo não afeta diretamente outro.



THREAD

- É uma “linha de execução” dentro de um processo.
- Compartilha memória com outras threads do mesmo processo.
- Mais leve para criar e trocar de contexto.

Exemplo: um navegador é um processo; cada aba pode ser uma thread (ou processo, dependendo do navegador).



SÍNTESE TÉCNICA

“Gerenciar processos é decidir, a cada instante, quem usa a CPU, por quanto tempo, e o que fazer quando ela precisa parar.”



09 / 09

Docência



DISCUSSÃO DIDÁTICA — PROFCOMP

Como Ensinar Escalonamento de Processos?



Analogias do dia a dia

Fila de banco (FCFS), fila preferencial (prioridade), revezamento em jogos (Round Robin).



Simulações visuais

Diagramas de Gantt no quadro, simulando o escalonador passo a passo com os alunos.



Dinâmicas em sala

Alunos representam processos físicos, com cartões de “quantum” e uma “CPU” entre eles.

Pergunta para discussão: qual analogia funcionaria melhor com a sua turma, e por quê?



10 / 09

Atividade em Duplas



REFLEXÃO

Reflexão



FECHAMENTO CONCEITUAL

O que Levamos desta Aula



O processador não “escolhe” sozinho — ele segue regras (algoritmos de escalonamento).



O Sistema Operacional é o “maestro” que organiza milhares de processos por segundo.



Entender escalonamento é entender como a multitarefa realmente funciona, por dentro.

“O computador não entende ‘espera’ — ele apenas alterna, no tempo certo, entre quem pode usar a CPU.”



BNCC - Computação

Habilidades Desenvolvidas

EM13CO03 — "Identificar o comportamento dos algoritmos no que diz respeito ao consumo de recursos como tempo de execução, espaço de memória e energia, entre outros."

EM13CO06 — "Avaliar software levando em consideração diferentes características e métricas associadas."